EECS 348 Assignment 4: UML Use Case Diagram

Sudoku solver (recursive backtracking) | Author: Nivedita Rekha | Diagram and text drafted with the help of Claude (Anthropic) and revised by the author.

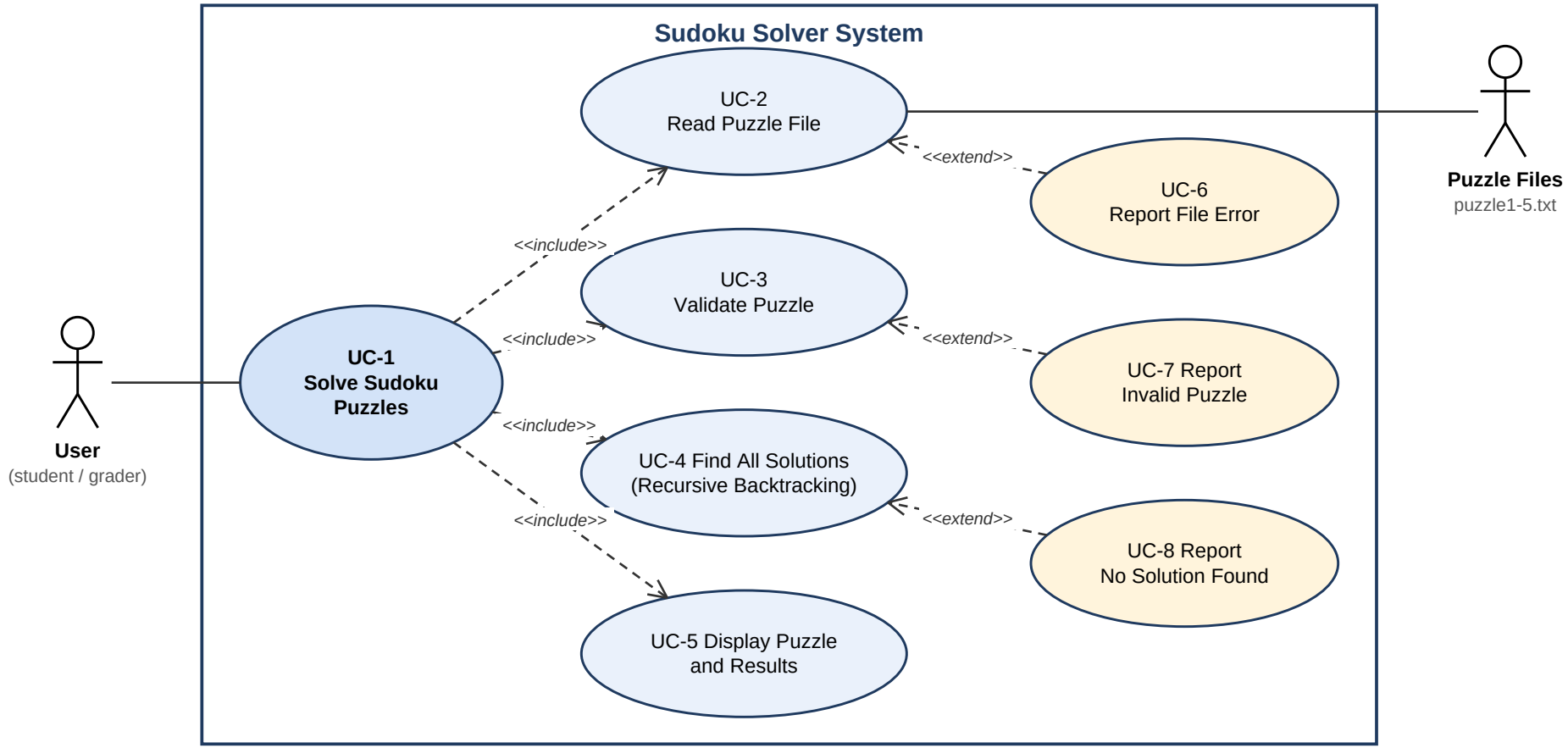

Figure 1. Use case diagram. Solid lines are associations between actors and use cases; dashed arrows are <<include>> (base to included) and <<extend>> (extension to base) relationships. Yellow use cases are exceptional/alternative behavior.

a. Identify / discover the use case

The assignment describes one thing a user wants from the system: given Sudoku puzzle files, get every solution (or a statement that none exists). That goal is the primary use case, **UC-1 Solve Sudoku Puzzles**. Looking at the required behavior revealed the smaller use cases it depends on (included) and the exceptional cases that only sometimes happen (extensions).

ID	Use case	Kind	Actors	Related to
UC-1	Solve Sudoku Puzzles	Primary	User	Includes UC-2, 3, 4, 5
UC-2	Read Puzzle File	Included	Puzzle Files	Extended by UC-6
UC-3	Validate Puzzle	Included	-	Extended by UC-7
UC-4	Find All Solutions (Recursive Backtracking)	Included	-	Extended by UC-8
UC-5	Display Puzzle and Results	Included	User	-
UC-6	Report File Error	Extension	User	Extends UC-2
UC-7	Report Invalid Puzzle	Extension	User	Extends UC-3
UC-8	Report No Solution Found	Extension	User	Extends UC-4

b. Brief description of the use case

UC-1 Solve Sudoku Puzzles. The user runs the program on the Cycle server. For each of the five puzzle files (puzzle1.txt through puzzle5.txt) the program reads the 9x9 grid, checks that it is a legal puzzle, and uses a recursive depth-first search with backtracking to find every solution. It prints the file name, the original puzzle, and each solution found, or "No solution found" if none exists.

c. Step-by-step flow of events (outline)

1. The User runs Assignment4.exe from a folder that contains puzzle1.txt through puzzle5.txt.

2. The system selects the next puzzle file, starting with puzzle1.txt.

3. The system reads the file into a 9x9 grid (UC-2). Digits are filled cells and underscores are blanks.

4. The system validates the puzzle (UC-3): exactly 81 legal cells, and no repeated digit in any row, column, or 3x3 block.

5. The system prints the file name and the puzzle as read (UC-5).

6. The system searches for solutions (UC-4): it picks an empty cell, computes its candidate digits, tries each one, and recurses.

7. If a cell has no candidates, the system backtracks, undoing its last choice and trying the next candidate.

8. Each time the grid is completely filled, the system prints that solution (UC-5) and keeps searching for more.

9. When the search is exhausted and no solution was found, the system prints "No solution found" (UC-8).

10. The system repeats steps 2-9 for puzzle2.txt through puzzle5.txt, then the program ends.

d. Detailed description of the use case

Use case name / ID	Solve Sudoku Puzzles (UC-1)
Primary actor	User (student running the program, or a GTA grading it on the Cycle server)
Secondary actor	Puzzle Files (puzzle1.txt through puzzle5.txt), the source of the input data
Description	Reads each Sudoku puzzle file, validates the puzzle, finds all of its solutions using recursive depth-first search with backtracking, and prints the puzzle and the results.
Trigger	The User runs Assignment4.exe.
Preconditions	• The program was compiled on the EECS Cycle server and runs there. • puzzle1.txt through puzzle5.txt are in the folder the program is run from. • Each file holds 81 cells: digits 1-9 for filled cells and _ for blank cells. Whitespace between cells is optional.
Postconditions	• For every readable file, the file name and the original puzzle have been printed. • Every solution of each valid puzzle has been printed, or "No solution found". • The input files are not modified.
Main success flow	1. The User runs Assignment4.exe from a folder that contains puzzle1.txt through puzzle5.txt. 2. The system selects the next puzzle file, starting with puzzle1.txt. 3. The system reads the file into a 9x9 grid (UC-2). Digits are filled cells and underscores are blanks. 4. The system validates the puzzle (UC-3): exactly 81 legal cells, and no repeated digit in any row, column, or 3x3 block. 5. The system prints the file name and the puzzle as read (UC-5). 6. The system searches for solutions (UC-4): it picks an empty cell, computes its candidate digits, tries each one, and recurses. 7. If a cell has no candidates, the system backtracks, undoing its last choice and trying the next candidate. 8. Each time the grid is completely filled, the system prints that solution (UC-5) and keeps searching for more. 9. When the search is exhausted and no solution was found, the system prints "No solution found" (UC-8). 10. The system repeats steps 2-9 for puzzle2.txt through puzzle5.txt, then the program ends.
Alternative flows	• **A1 - Several solutions (step 8):** the search does not stop at the first solution. It prints each solution as it is found, then backtracks and continues until all candidates are exhausted. • **A2 - Puzzle with no blanks (step 6):** the grid is already complete, so it is printed as the single solution if it is valid.
Exception flows	• **E1 - File error (UC-6, extends UC-2):** the file is missing, contains a character other than 1-9, _ or whitespace, or does not contain exactly 81 cells. The system prints an error message for that file and continues with the next file. • **E2 - Invalid puzzle (UC-7, extends UC-3):** the given digits already repeat in a row, column, or block. The system prints the puzzle and "No solution found" without searching. • **E3 - No solution (UC-8, extends UC-4):** the search ends without ever filling the grid. The system prints "No solution found".
Includes	UC-2 Read Puzzle File, UC-3 Validate Puzzle, UC-4 Find All Solutions, UC-5 Display Puzzle and Results
Extended by	UC-6 Report File Error, UC-7 Report Invalid Puzzle, UC-8 Report No Solution Found

Business rules	• Each digit 1-9 appears exactly once in every row, column, and 3x3 block. • A puzzle may have more than one solution; all must be printed. • The solution is found by brute-force depth-first search with backtracking, not by logical deduction.
Special requirements	• Object-oriented design (classes, not just functions) in a single file, Assignment4.cpp. • Output for each file: file name, puzzle, then solution(s) or "No solution found". • Hard puzzles should finish in a small fraction of a second. • Memory use should not grow with the number of solutions (print each solution as found).
Assumptions	• The user does not interact with the program after starting it. • Input is read only from the five fixed file names.